哈爾濱工程大學

第 04 讲 / 全 17 讲

C 语言 switch 语句与多分支详解

把长长的 else if 阶梯，换成一个清晰的开关

switch

先算出一个整数值

再决定往哪走

case

一个个常量入口

匹配上了就进来

break

干完活立刻跳出去

漏写就会穿透

default

谁都没匹配上的兜底

建议总是写上

本节目标

哈爾濱工程大學

1. 认识 switch 的四个零件：表达式、 case 、 break 、 default
2. 独立写出 switch-case 骨架，把 else if 阶梯改写成 switch
3. 记住 case 后只能是整型或字符型常量，不能是变量或浮点
4. 讲清 break 的作用，一眼看穿 " 漏写 break 造成穿透 " 的错误
5. 用 default 兜底，并会用穿透让多个 case 共用一段代码
6. 独立完成 5 道例题：星期、等级、计算器、月天数、菜单

01

switch 基础：把阶梯换成开关

switch 的语法骨架 · case 常量 · break 跳出 · default 兜底 · 穿透陷阱

哈爾濱工程大学

else if 阶梯写长了很累

```
// 第 03 讲的老办法：一个一个 else if
int day;
scanf("%d", &day);
if (day == 1) printf("星期一");
else if (day == 2) printf("星期二");
else if (day == 3) printf("星期三");
else if (day == 4) printf("星期四");
else if (day == 5) printf("星期五");
else if (day == 6) printf("星期六");
else if (day == 7) printf("星期日");
else printf("输入有误");
```

写得累

7 个分支要写 7 次判断，
再加一档就要再补一行。

看得累

每行都在重复 day == 这半句，
真正有用的只有后面的数字。

switch 更省心

先算出 day 的值，再让各个分支
对号入座，一眼看清有哪几档。

switch 的语法骨架

```
// switch 骨架：四个零件
switch (day)          // ① 表达式
{
    case 1:           // ② 常量入口
        printf("星期一");
        break;        // ③ 跳出 switch
    case 2:
        printf("星期二");
        break;
    default:          // ④ 都不匹配时
        printf("输入有误");
}
```

① switch（表达式）

先算出这个表达式的值，只能是整型或字符型，浮点一律不许。

② case 常量：

表达式的值等于这个常量，就从这里进入，一直往下执行。

③ break;

干完活立刻跳出整个 switch ，后面的 case 一律不再看。

④ default:

上面所有 case 都没匹配上时才走到，写不写都不报错，但建议总写上。

case 后面能写什么

```c
// ✓ 合法：整型常量 / 字符常量
switch (x)
{
    case 1:   printf("一"); break;
    case 10:  printf("十"); break;
    // 字符常量，本质也是整数
    case 'A': printf("优秀"); break;
    case 'a': printf("良好"); break;
    default:  printf("其他"); break;
}
```

```c
// ✗ 以下写法都不合法
switch (x)
{
    case n:       // 变量 → 不行
    case 3.14:    // 浮点 → 不行
    case a + 1:   // 含变量 → 不行
    default: ;
}
// 想写范围？case > 5: 没这个语法
// 范围判断请回到 if-else 家族
```

一句话记住

case 后面必须是整型或字符型的常量（ int 、 char 都行），
不能是变量、不能是小数，更不能写成范围——那是 if-else 的活。

break 的作用：跳出 switch

```
// 每个 case 的最后都写上 break
switch (day)
{
    case 1:
        printf("星期一");
        break;      // 到此为止，出去
    case 2:
        printf("星期二");
        break;
    default:
        printf("输入有误");
}
```

遇到 break 立刻跳出

day 是 1 时：打印"星期一"后
遇到 break ，直接跳到 switch 之后，
case 2 、 default 全都不看。

没有 break 就继续往下

少了 break ，程序会一路穿过后面的
case ，把它们的语句也执行掉，
这就是"穿透" ，下一页细讲。

break 属于 switch

这里的 break 只跳出 switch 这一层，
不是跳出整个程序； switch 后面的
语句照常执行。

漏掉 break 就穿透了

本讲第一大坑

哈爾濱工程大學

```
// 三个 case 全都没写 break
int day = 2;
switch (day)
{
    case 1:
        printf("星期一");
    case 2:
        printf("星期二");
    case 3:
        printf("星期三");
    default:
        printf("输入有误");
}
```

逐行追踪执行流程

① day 的值是 2 → 直接跳到 case 2

② 打印“星期二”（这一步是对的）

③ 没有 break → 继续往下穿透

④ 打印“星期三”（本不该执行）

⑤ 打印“输入有误”（本不该执行）

⑥ 一直穿到 switch 结尾才停

实际输出

星期二星期三输入有误

// 三句连在一起，全跑了

default 分支：兜住所有意外

```
// default：所有 case 都没匹配上时走
int day;
scanf("%d", &day);
switch (day)
{
    case 1: printf("星期一"); break;
    case 2: printf("星期二"); break;
    case 3: printf("星期三"); break;
    default: printf("只认 1~3"); break;
}
// 输入 9 → 输出：只认 1~3
```

谁都没匹配上才走它

1、2、3 都对不上号时，程序自动落到 default，给出一句提示，而不是什么都不做。

可以省略，但别省

语法上 default 不是必须的；可一旦省掉，用户输错值时程序一声不吭，这种 bug 最难查。

习惯上放最后

default 写在哪都能跑，但放最后最好读；如果放中间或开头，一定要给它配上 break。

02

深入理解：穿透、选型与字符

穿透的正当用法 · default 与 break 的位置 · switch 与 if-else 怎么选 · 字符型 switch · 菜单雏形

哈爾濱工程大学

穿透的正当用法：多 case 共用

同一个坑，换个角度就是技巧

哈尔滨工程大学

```
// A、B、C 三档都算及格 → 共用一段
switch (grade)
{
    case 'A':
    case 'B':
    case 'C':
        printf("及格");
        break;       //  只在这收口
    default:
        printf("非法等级");
        break;
}
```

为什么能这么写

case 只是一个"入口"，进去之后
一路往下跑，遇到 break 才停下来。
所以空 case 就是"共用下一份代码"。

好处：不抄三遍

不用把同一句 printf 复制三次，
改需求时只改一处，漏改的风险
也小得多。

末尾的 break 别省

共用代码段的最后一句必须 break ，
否则会一路穿到 default 去，
又变成上一页那个 bug 。

case 的顺序与 break 的位置


```c
// 顺序不影响匹配，但影响阅读
int n;
scanf("%d", &n);
switch (n)
{
    case 1: printf("一"); break;
    case 2: printf("二"); break;
    case 3: printf("三"); break;
    default: printf("其他"); break;
}
// 写乱序也能编译，但读起来难受
```

顺序：谁先谁后都能跑

switch 是直接"对号入座"，不是从上往下比；但按 1、2、3 排序写，人和机器都舒服。

default 放最后

这是约定俗成的写法；放中间也能编译，但读者要去找它，还容易漏掉它的 break。

break 放在该 case 末尾

break 要写在这个 case 所有语句的最后一行；写早了，后面的语句永远执行不到。

switch 里写变量声明要注意

本讲只提一句，不深入

哈尔滨工程大学

一句话结论

初学阶段，变量一律在 switch 之前就声明好，case 里面只做赋值和打印，
别在 case 里临时声明新变量——理由等第 06 讲讲作用域时自然明白。

```
// 推荐写法：先声明，再在 case 里赋值
int n, result;
scanf("%d", &n);
switch (n)
{
    case 1: result = 10; break;
    case 2: result = 20; break;
    default: result = 0; break;
}
printf("%d", result);
```

为什么先声明

所有 case 共享 switch 后面那一对花括号，在某一个 case 里声明的变量，别的分支"看得见却可能没执行到"，很容易写出怪 bug。

本讲的做法

需要什么变量，就在 switch 前面一排声明完；case 里只管赋值。

真要在分支里加花括号，等学完作用域再说。

switch 与 if-else 怎么选

选型对照表

对比项	switch	if-else 家族
判断依据	一个表达式的整型值	任意条件，只要能判真假
分支条件	只能写 " 等于某个常量 "	可以写大于、小于、区间
分支变多时	结构清爽，一屏看清有哪几档	阶梯越写越长，读起来费劲
典型陷阱	漏写 break 就穿透	没有 break 这回事
什么时候用	**离散值多选一 → 选它**	**范围判断 → 选它**

一句话：值能一个一个数出来（ 1 、 2 、 3 、菜单选项）就用 switch ；要判断区间就用 if-else 。

字符型 switch ：用字母做选项

```
// char 本质是整数，可以直接上 switch
char ch;
scanf("%c", &ch);
switch (ch)
{
    case 'A': printf("增加"); break;
    case 'B': printf("删除"); break;
    case 'C': printf("查询"); break;
    default:  printf("无效选项"); break;
}
// 输入 A → 输出：增加
```

怎么读进来的

用 char 变量配 scanf("%c", &ch) 读一个字符，这在第 02 讲就学过，换到 switch 里一样好用。

单引号不能丢

字符常量必须带单引号；写成 case A 会被当成变量名，直接报错。双引号是字符串，也不能用。

大小写是两回事

A 和 a 的 ASCII 码不同，是两个不同的常量；想都认，就用两个 case 共用一段代码。

菜单程序雏形

printf + scanf + switch 三件套

哈尔滨工程大学

```c
// 菜单三步：打印 → 读选项 → 分派
int choice;
printf("1.查询  2.修改  0.退出  ");
scanf("%d", &choice);
switch (choice)
{
    case 1: printf("执行查询"); break;
    case 2: printf("执行修改"); break;
    case 0: printf("再见"); break;
    default: printf("没有这项"); break;
}
// 输入 2 → 输出：执行修改
```

第一步：printf 打菜单

把能选的项一行行打印出来，
每项前面配一个数字编号，
让用户知道该按几。

第二步：scanf 读选项

用 int 接住用户输入的编号，
这个编号正好是离散值，
天生适合 switch。

现在只能执行一次

循环还没学，选完一次程序就结束。
第 05 讲学了 for 之后，把它包进
循环，就是真正的菜单了。

03

综合应用：技巧与完整案例

break 的完整规则 · 嵌套 switch · 范围映射技巧 · switch 里的 return · 计算器综合案例

哈爾濱工程大学

break 跳的是 switch ，不是程序

```
// break 只跳出 switch，不是退出程序
int n = 1;
switch (n)
{
    case 1:
        printf("选中 1");
        break;   // 从这里跳出去
    case 2:
        printf("选中 2");
        break;
    default:
        printf("其他");
}
printf("switch 之后");  // 一定会执行
```

落点在哪

break 的落点是 switch 结束的那个右花括号之后，也就是整个 switch 结构之外。

后面的话照常说

switch 之后的 printf、return 0 等语句一律照常执行，一条都不少，输出连在一起就是这样。

想直接结束程序？

break 做不到，得用 return ；这两个词长得像，干的事完全不一样，下一页就讲它。

嵌套 switch：switch 里再套

看得懂就行，不要求写

```
// 外层挑大类，内层挑小类
switch (grade)
{
    case 1:
        switch (cls)
        {
            case 1: printf("1 班"); break;
            case 2: printf("2 班"); break;
        }
        break;      // 外层还要收口
    case 2:
        printf("二年级"); break;
    default: break;
}
```

结构：套在 case 里

内层 switch 要完整地写在外层某个 case 的语句块里，该有的 case、break、default 一个都不能少。

break 只管一层

内层的 break 只跳出内层 switch，跳完还得靠外层那个 break 收口；漏了它就会穿到 case 2 去。

实际很少这么写

两层已经够难读了；真遇到"先分大类再分小类"，多数人会写成嵌套 if，或者等学了函数再拆。

用 switch 处理范围值：先映射

score / 10 是经典套路

哈尔滨工程大学 1953

```
// 0~100 的范围 → 0~10 的离散值
int score;
scanf("%d", &score);
switch (score / 10)
{
    case 10:
    case 9:  printf("A"); break;
    case 8:  printf("B"); break;
    case 7:  printf("C"); break;
    case 6:  printf("D"); break;
    default: printf("E"); break;
}
// 85 → 8 → B ;   100 → 10 → A
```

为什么除以 10

整数除法自动舍掉个位：85/10 得 8，100/10 得 10。范围就这样被压成一个个能写进 case 的离散值。

case 10 和 case 9 共用

100 分除以 10 是 10，90~99 是 9，两档都是 A，就让 case 10 空着穿透到 case 9 —— 第 11 页的技巧。

default 兜住不及格

0~59 分除以 10 是 0~5，全都落进 default 输出 E；省掉 default 的话不及格就没有任何输出。

switch 里的 return ：直接收工

提前认识，第 06 讲正式讲

哈尔滨工程大学

```c
// return 出现在 case 里：整个函数结束
switch (n)
{
    case 0:
        printf("退出");
        return 0;    // 后面全不算数
    default:
        printf("继续");
        break;
}
printf("收尾工作");  // 输入 0 时看不到
```

return 比 break 更狠

break 只是跳出 switch，后面的话说照说；return 是直接结束整个函数，回到调用它的地方去。

用了 return 就不用 break

执行到 return 人已经走了，后面再写 break 也执行不到；不过写上也不算错，图个安心。

本讲不要求掌握

函数要到第 06 讲才学，这里只是让你先见一面：return 表示"这个函数到此结束，把结果交回去"。

综合案例：四则运算计算器

字符型 switch + 除零检查

哈尔滨工程大学

```c
// 输入形如 8+2 的式子，输出运算结果
double a, b;
char op;
scanf("%lf%c%lf", &a, &op, &b);
switch (op)
{
    case '+': printf("%.2f", a + b); break;
    case '-': printf("%.2f", a - b); break;
    case '*': printf("%.2f", a * b); break;
    case '/': printf("%.2f", a / b); break;
    default: printf("不支持的运算符");
}
```

陷阱：除数可能是 0

8 / 0 在数学上没有意义，程序真这么算会当场出问题，必须先用 if 挡一道。

```c
// 除号这一档改成这样
case '/':
    if (b != 0) printf("%.2f", a / b);
    else printf("除数不能为 0");
    break;   // break 留在 if 外面
```

运行示例

输入 8+2 → 输出 10.00

输入 8/0 → 输出 除数不能为 0

输入 8?2 → 输出 不支持的运算符

switch 代码风格建议


```
// 好风格：对齐清楚、每个分支都收口
switch (op)
{
    case '+':
        result = a + b;
        break;
    case '-':
        result = a - b;
        break;
    default:
        result = 0;   // 兜底也要有
        break;
}
```

case 对齐

case 相对 switch 缩进一级，
case 里的语句再缩进一级；
全篇统一，别一个紧一个松。

每个 case 都写 break

包括最后一个和 default 也写上；
以后往中间插新 case 时，
不会因为少一个 break 出 bug。

一个 case 只干一件事

别在一个 case 里塞十几行；真要
写很多，就说明该考虑别的写法了
（比如等第 06 讲把它拆成函数）。

例 1 ：输入 1-7 输出星期几

哈爾濱工程大學

例 1 输入 1~7 之间的整数，输出对应的星期几；输入其他数字给出提示　样例： 3 → 星期三

```
//  离散值选一 → 最典型的  switch
int main(void)
{
    int day;
    scanf("%d", &day);
    switch (day)
    {
        case 1: printf(" 星期一 "); break;
        case 2: printf(" 星期二 "); break;
        case 3: printf(" 星期三 "); break;
        case 4: printf(" 星期四 "); break;
        case 5: printf(" 星期五 "); break;
```

```
//  接上：剩下的两档和兜底
        case 6: printf(" 星期六 "); break;
        case 7: printf(" 星期日 "); break;
        default: printf(" 输入有误 "); break;
    }
    return 0;
}
```

运行结果

输入 3 → 输出 星期三

输入 9 → 输出 输入有误（靠 default ）

七档全是离散值，用 switch 最清爽

例 2：百分制转等级 A-E

哈爾濱工程大學

例 2 输入百分制成绩，输出等级：90 以上 A，80~89 B，70~79 C，60~69 D，其余 E　　样例：85，B

```
// 范围 → 离散：先除以 10
int main(void)
{
    int score;
    scanf("%d", &score);
    switch (score / 10)
    {
        case 10:
        case 9:  printf("A"); break;
        case 8:  printf("B"); break;
        case 7:  printf("C"); break;
```

```
// 接上：D 档与兜底
        case 6:  printf("D"); break;
        default: printf("E"); break;
    }
    return 0;
}
```

运行结果与要点

输入 85 → 8 → 输出 B

输入 100 → 10 → 输出 A

输入 59 → 5 → 输出 E（走 default）

case 10 空着穿透到 case 9，是故意的

范围判断的活，靠 /10 变成了离散值

例 3：简单四则运算计算器

哈尔滨工程大学

例 3 输入形如 8*2 的算式，输出运算结果，保留两位小数；除数为 0 要提示。样例：8*2 → 16.00

```
// 运算符是 char，天然适合 switch
int main(void)
{
    double a, b;
    char op;
    scanf("%lf%c%lf", &a, &op, &b);
    switch (op)
    {
        case '+': printf("%.2f", a + b); break;
        case '-': printf("%.2f", a - b); break;
        case '*': printf("%.2f", a * b); break;
        case '/':
```

```
// 接上：除法要先挡住 0
            if (b != 0) printf("%.2f", a / b);
            else printf("除数不能为 0");
            break;
        default: printf("不支持的运算符");
    }
    return 0;
}
```

运行结果

输入 8*2 → 输出 16.00

输入 8/0 → 输出 除数不能为 0

break 写在 if 外面，两个分支都能跳出

例 4 ：输入月份输出该月天数

例 4 输入年份和月份，输出该月天数（ 2 月按闰年判断，非法月份输出 0 ）样例： 2024 2 → 29

```
// 多 case 共用: 31 天组、 30 天组
int main(void)
{
    int year, month, days;
    scanf("%d%d", &year, &month);
    switch (month)
    {
        case 1: case 3: case 5: case 7:
        case 8: case 10: case 12:
            days = 31; break;
        case 4: case 6: case 9: case 11:
            days = 30; break;
```

```
// 2 月交给嵌套 if 处理闰年
        case 2:
            if (year % 400 == 0) days = 29;
            else if (year % 100 == 0) days = 2
            else if (year % 4 == 0) days = 29;
            else days = 28;
            break;
        default: days = 0; break;
    }
    printf("%d", days);
    return 0;
}
```

例 5 ：菜单选择程序

哈爾濱工程大學 1953 HARBIN ENGINEERING UNIVERSITY

例 5 打印菜单，读入选项： 1 查询、 2 修改、 0 退出，其他数字给出提示 样例： 2 → 执行修改

```
// 打印菜单 → 读选项 → switch 分派
int main(void)
{
    int choice;
    printf("1. 查询  2. 修改  0. 退出  ");
    scanf("%d", &choice);
    switch (choice)
    {
        case 1: printf(" 执行查询 "); break;
        case 2: printf(" 执行修改 "); break;
        case 0: printf(" 已退出 "); break;
```

```
// 接上：兜底与收尾
        default: printf(" 没有这个选项 "); break;
    }
    return 0;
}
```

运行结果与延展

屏幕先显示： 1. 查询 2. 修改 0. 退出

输入 1 → 执行查询 ；输入 0 → 已退出

输入 9 → 没有这个选项（ default 接住 ）

第 05 讲学了 for 循环，把它包进去就是真菜单

现在只能选一次，选完程序就结束

常见错误盘点

这 6 条考试最常考

哈尔滨工程大学

✗ case 后忘写 break → 一路穿透，把后面所有 case 的语句都执行了	每个 case 都补上 break
✗ case 后写范围判断 —— 根本没有这种语法	范围判断交给 if-else
✗ switch 的表达式用了浮点型，编译就过不去	只认整型和字符型
✗ case 常量写重复了，同一个值出现两次	编译器会直接报错
✗ default 忘写 → 输入非法值时程序一声不吭	兜底分支建议总写上
✗ 把 break 写进 if 里 → 条件不成立时照样穿透	break 要留在 if 外面

本节小结

哈爾濱工程大学

switch 是 " 对号入座 " ：值能一个一个数出来时，它比 else if 阶梯清爽得多。

case 只认整型或字符型常量， break 负责收口， default 负责兜底。

漏写 break 会穿透——它是本讲第一大坑，也是多个 case 共用代码的技巧。

下节预告

第 05 讲 for 循环 —— 让计算机替你把重复的事做上一万遍

到那时，今天这个只能选一次的菜单，就能一直选下去了。